

Gradient descent

Fraida Fund

Contents

In this lecture	2
Runtime of OLS solution	2
Limitations of OLS solution	2
Background: Big O notation	2
Computing OLS solution	2
Solution using gradient descent	3
Iterative solution	3
Background: Gradients and optimization	3
Gradient descent idea	3
Standard (“batch”) gradient descent	4
Example: gradient descent for linear regression (1)	4
Example: gradient descent for linear regression (2)	4
Variations on main idea	7
Stochastic gradient descent	7
Mini-batch (also “stochastic”) gradient descent (1)	8
Mini-batch (also “stochastic”) gradient descent (2)	8
Selecting the learning rate	9
Annealing the learning rate	9
Gradient descent in a ravine (1)	10
Gradient descent in a ravine (2)	10
Momentum (1)	11
Momentum (2)	11
Momentum: pseudocode	11
Momentum: illustrated	11
AdaGrad (1)	12
AdaGrad (2)	12
AdaGrad: pseudocode	12
RMSProp: Leaky AdaGrad	12
RMSProp: pseudocode	13
Adam: Adaptive Moment Estimation	13
Adam: pseudocode vs Momentum	13
Adam: pseudocode vs RMSProp	14
Adam: Pseudocode with bias correction	14
Illustration (Beale’s function)	15
Illustration (Long valley)	15
Recap	15

Math prerequisites for this lecture: You should know about:

- derivatives and optimization (Appendix C in Boyd and Vandenberghe)
- complexity of algorithms and especially of vector and matrix operations (Appendix B in Boyd and Vandenberghe, also the complexity part of Section I, Chapter 1 and Section II, Chapter 5)

In this lecture

Addresses “What is the cost of training/inference?”

- Runtime of OLS solution for multiple/LBF regression
- Solution using gradient descent
- Variations on main idea

Runtime of OLS solution

Limitations of OLS solution

- Specific to linear regression, L2 loss
- For extremely large datasets: runtime, memory

Background: Big O notation

Approximate the number of operations required, as a function of input size.

- Ignore constant terms, constant factors
- Ignore all but the dominant term

Example: $3n^3 + 100n^2 + 1000$ would be $O(n^3)$.

Computing OLS solution

How long does it take to compute

$$\mathbf{w}^* = (\Phi^T \Phi)^{-1} \Phi^T \mathbf{y}$$

where Φ is an $N \times d$ design matrix with d columns, including any constant basis column?

Runtime of a “naive” solution using “standard” matrix multiplication:

- $O(d^2 N)$ to multiply $\Phi^T \Phi$
- $O(dN)$ to multiply $\Phi^T \mathbf{y}$
- $O(d^3)$ to compute the inverse of $\Phi^T \Phi$ (Note: in practice, we can do it a bit faster.)

Since N is generally much larger than d , the first term dominates and the runtime is $O(d^2 N)$.

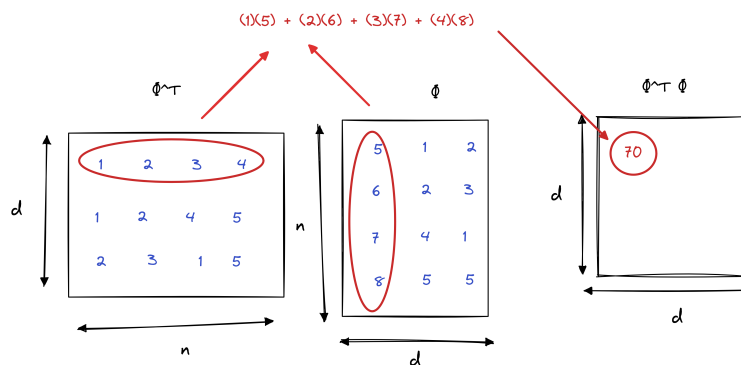


Figure 1: Computing $\Phi^T \Phi$. For each entry in the matrix, we need N multiplications, and then we need to fill in $d \times d$ entries to complete the matrix.

Solution using gradient descent

Iterative solution

Suppose we would start with all-zero or random weights. Then iteratively (for t rounds):

- pick random weights
- if loss performance is better, keep those weights
- if loss performance is worse, discard them

For infinite t , we'd eventually find optimal weights - but clearly we could do better.

Background: Gradients and optimization

Gradient has two important properties for optimization:

At a minima (or maxima, or saddle point),

$$\nabla L(\mathbf{w}) = 0$$

At other points, $\nabla L(\mathbf{w})$ points towards direction of maximum (infinitesimal) rate of *increase*.

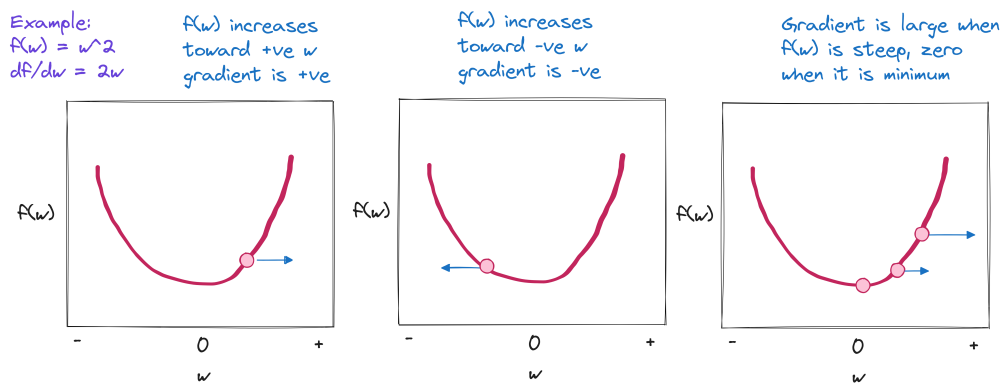


Figure 2: Properties of gradient.

Gradient descent idea

To move towards minimum of a (smooth, convex) function:

Start from some initial point, then iteratively

- compute gradient at current point, and
- add some fraction of the **negative** gradient to the current point

Standard (“batch”) gradient descent

For each step t along the error curve:

$$\begin{aligned}\mathbf{w}^{t+1} &= \mathbf{w}^t - \alpha \nabla L(\mathbf{w}^t) \\ &= \mathbf{w}^t - \frac{\alpha}{N} \sum_{i=1}^N \nabla \ell_i(\mathbf{w}^t, \mathbf{x}_i, y_i)\end{aligned}$$

Repeat until stopping criterion is met.

“Stopping criteria” may be: loss is sufficiently small, gradient is sufficiently close to zero, or a pre-set max number of iterations is reached.

Note: the superscript t tracks what iteration we are on. It’s not an exponent!

Example: gradient descent for linear regression (1)

With an intercept, define $\mathbf{a}_i = [1, \mathbf{x}_i^T]^T \in \mathbb{R}^{d+1}$ and let $\mathbf{A} \in \mathbb{R}^{N \times (d+1)}$ contain one $\mathbf{a}_i^T$ per row. The mean squared error is

$$\begin{aligned}L(\mathbf{w}) &= \frac{1}{N} \sum_{i=1}^N (y_i - \mathbf{w}^T \mathbf{a}_i)^2 \\ &= \frac{1}{N} \|\mathbf{y} - \mathbf{A}\mathbf{w}\|^2\end{aligned}$$

Gradient of the loss function is:

- Vector form: $\nabla L(\mathbf{w}^t) = \frac{-2}{N} \sum_{i=1}^N (y_i - (\mathbf{w}^t)^T \mathbf{a}_i) \mathbf{a}_i$
- Matrix form: $\nabla L(\mathbf{w}^t) = \frac{-2}{N} \mathbf{A}^T (\mathbf{y} - \mathbf{A}\mathbf{w}^t)$

The constant factor 2 can be absorbed into the learning rate. We move in the *opposite* direction, so...

Example: gradient descent for linear regression (2)

We will compute the weights at each step as

$$\begin{aligned}\mathbf{w}^{t+1} &= \mathbf{w}^t + \frac{\alpha^t}{N} \sum_{i=1}^N (y_i - (\mathbf{w}^t)^T \mathbf{a}_i) \mathbf{a}_i \\ &= \mathbf{w}^t + \frac{\alpha^t}{N} \mathbf{A}^T (\mathbf{y} - \mathbf{A}\mathbf{w}^t)\end{aligned}$$

(with the constant factor 2 absorbed into α^t)

To derive the gradient descent update rule, write the loss as a sum over training examples:

$$L(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N (y_i - \mathbf{w}^T \mathbf{a}_i)^2.$$

Take the partial derivative with respect to one weight w_j . Apply the chain rule to each squared residual:

$$\begin{aligned} \frac{\partial L(\mathbf{w})}{\partial w_j} &= \frac{1}{N} \sum_{i=1}^N \frac{\partial}{\partial w_j} (y_i - \mathbf{w}^T \mathbf{a}_i)^2 \\ &= \frac{1}{N} \sum_{i=1}^N 2 (y_i - \mathbf{w}^T \mathbf{a}_i) \frac{\partial}{\partial w_j} (y_i - \mathbf{w}^T \mathbf{a}_i) \\ &= \frac{1}{N} \sum_{i=1}^N 2 (y_i - \mathbf{w}^T \mathbf{a}_i) (-a_{ij}) \\ &= -\frac{2}{N} \sum_{i=1}^N (y_i - \mathbf{w}^T \mathbf{a}_i) a_{ij}. \end{aligned}$$

The derivative of the prediction is a_{ij} because

$$\mathbf{w}^T \mathbf{a}_i = \sum_{k=0}^d w_k a_{ik}, \quad \frac{\partial}{\partial w_j} (\mathbf{w}^T \mathbf{a}_i) = a_{ij}.$$

Collect the partial derivatives for all weights into a vector. The entries a_{ij} become the components of $\mathbf{a}_i$:

$$\nabla L(\mathbf{w}) = -\frac{2}{N} \sum_{i=1}^N (y_i - \mathbf{w}^T \mathbf{a}_i) \mathbf{a}_i.$$

At iteration t , substitute $\mathbf{w}^t$ into this gradient and move in the opposite direction:

$$\begin{aligned} \mathbf{w}^{t+1} &= \mathbf{w}^t - \alpha^t \nabla L(\mathbf{w}^t) \\ &= \mathbf{w}^t + \frac{2\alpha^t}{N} \sum_{i=1}^N (y_i - (\mathbf{w}^t)^T \mathbf{a}_i) \mathbf{a}_i. \end{aligned}$$

The factor 2 can be absorbed into the learning rate. Relabeling $2\alpha^t$ as α^t gives the summation update.

To update $\mathbf{w}$, we must compute N loss functions and gradients. With the intercept column, **each iteration** is $O(N(d+1)) = O(Nd)$. We need multiple iterations, but in many cases it's more efficient than the previous approach.

However, if N is large, it may still be expensive!

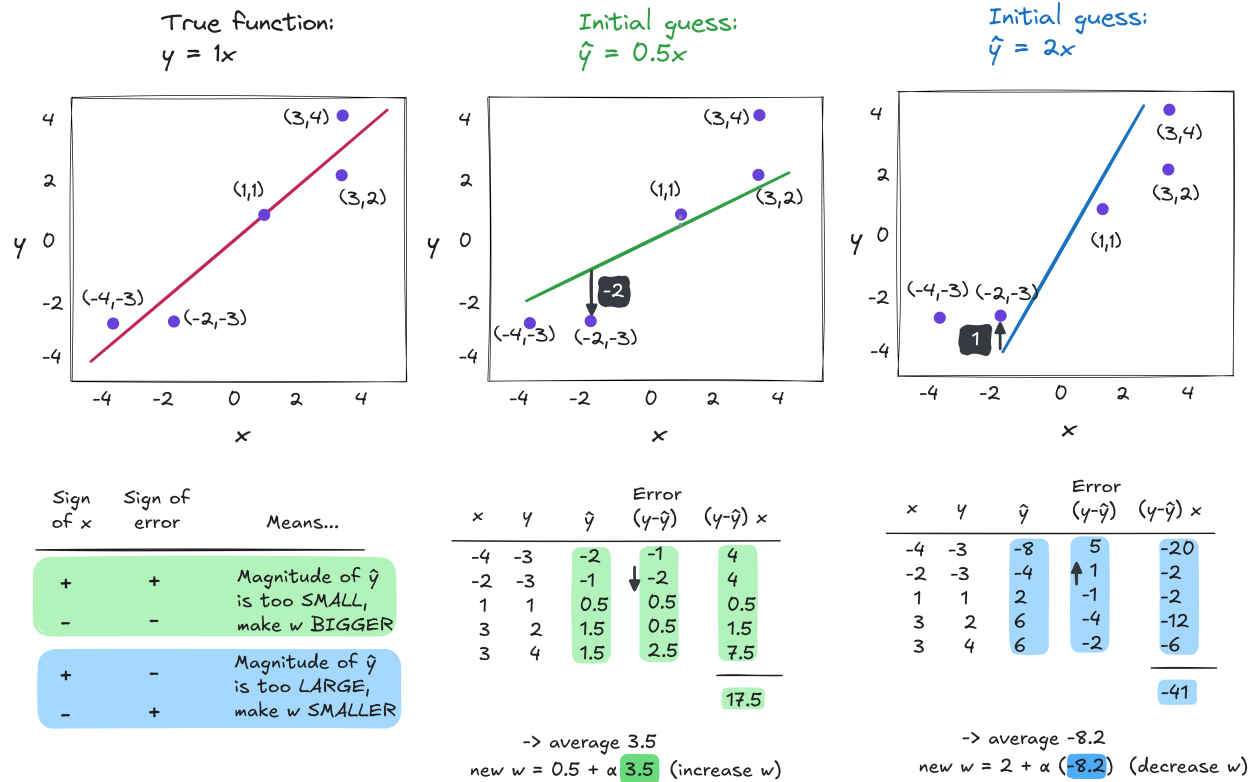


Figure 3: Example: gradient descent for linear regression, with too-small and too-big initial guesses.

Variations on main idea

Two main “knobs” to turn:

- “batch” size
- learning rate

Stochastic gradient descent

Idea:

At each step, compute estimate of gradient using only one randomly selected sample, and move in the direction it indicates.

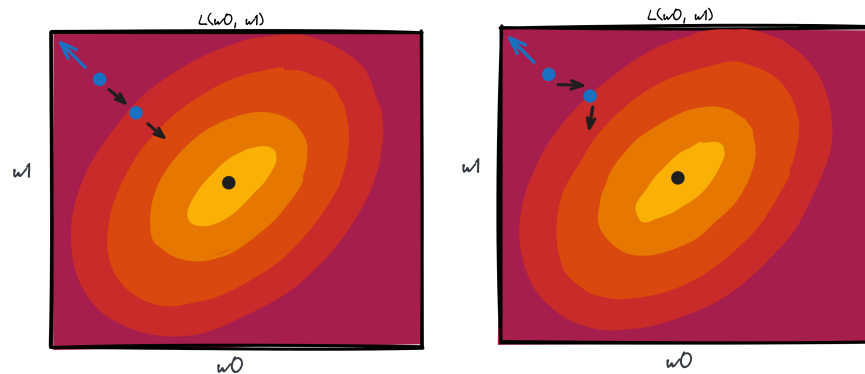


Figure 4: Full-batch gradient descent (left), SGD (right). Many of the steps will be in the wrong direction, but progress towards minimum occurs *on average*, as long as the steps are small.

Each iteration is now only $O(d)$, but we may need more iterations than for gradient descent. However, in many cases we still come out ahead (especially if N is large!).

See [supplementary notes](#) for an analysis of the number of iterations needed.

Also:

- SGD is often more efficient because of *redundancy* in the data - data points have some similarity.
- But, we miss out on the benefits of vectorization. In practice, it takes longer to compute something over 1 sample 1024 times, than over 1024 samples 1 time.
- If the function we want to optimize does not have a global minimum, the noise can be helpful - we can “bounce” out of a local minimum.

Mini-batch (also “stochastic”) gradient descent (1)

Idea:

At each step, select a small subset of training data (“mini-batch”), and evaluate gradient on that mini-batch.

Then move in the direction it indicates.

Mini-batch (also “stochastic”) gradient descent (2)

For each step t along the error curve:

- Select random mini-batch $I_t \subset 1, \dots, N$
- Compute gradient approximation:

$$g^t = \frac{1}{|I_t|} \sum_{i \in I_t} \nabla \ell(\mathbf{x}_i, y_i, \mathbf{w}^t)$$

- Update parameters: $\mathbf{w}^{t+1} = \mathbf{w}^t - \alpha^t g^t$

Now that each iteration is not equal to an iteration over *all* data, we need to introduce the idea of an “epoch”:

- One epoch = one pass over *all* the data
- Mini-batch SGD is often used in practice because we get some benefit of vectorization, but also take advantage of redundancy in data.

After a fixed number of epochs (passes over the entire data), * we may end up at a better minimum (lower loss) with a small batch size, * *but*, the time per epoch may be longer with a small batch size.

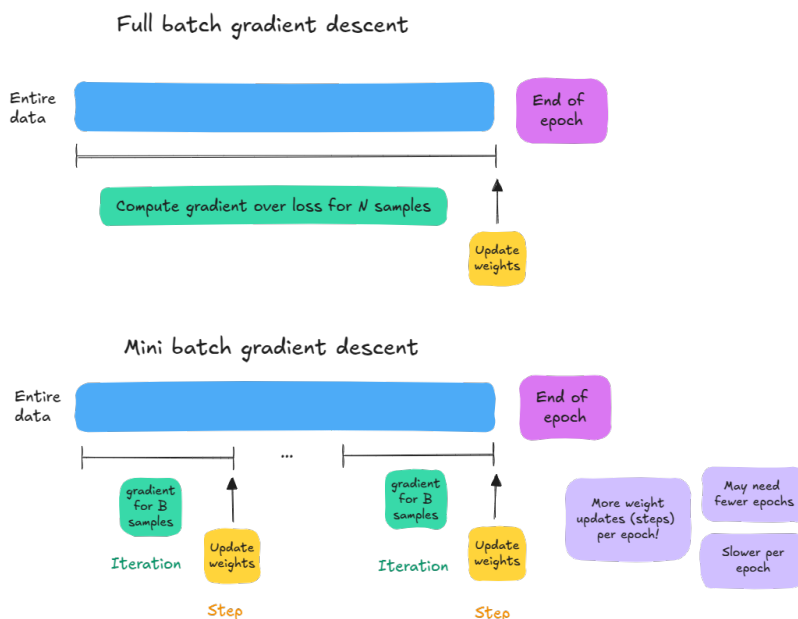


Figure 5: Comparison of full batch and mini batch over one epoch.

Selecting the learning rate

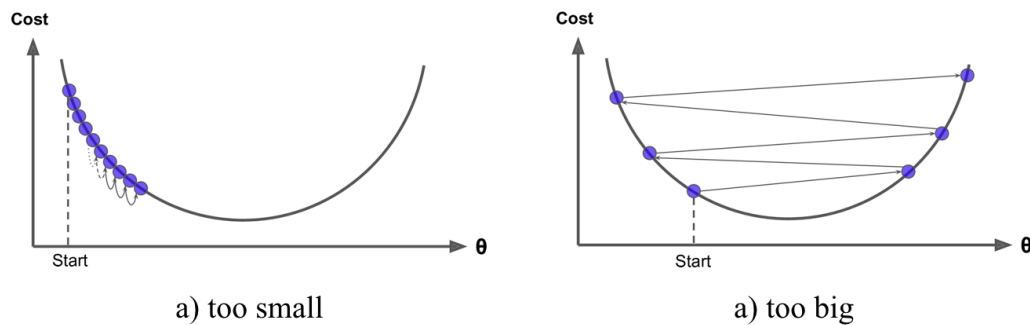


Figure 6: Choice of learning rate α is critical

Image credit: Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 2nd Edition, by Aurélien Géron.

Also note: SGD “noise ball”

Annealing the learning rate

One approach: decay learning rate slowly over time, such as

- Exponential decay: $\alpha^t = \alpha_0 e^{-kt}$
- 1/t decay: $\alpha^t = \alpha_0 / (1 + kt)$

(where k is tuning parameter).

But: this is still sensitive, requires careful selection of gradient descent parameters for the specific learning problem.

Can we do this in a way that is somehow “tuned” to the shape of the loss function?

Gradient descent in a ravine (1)

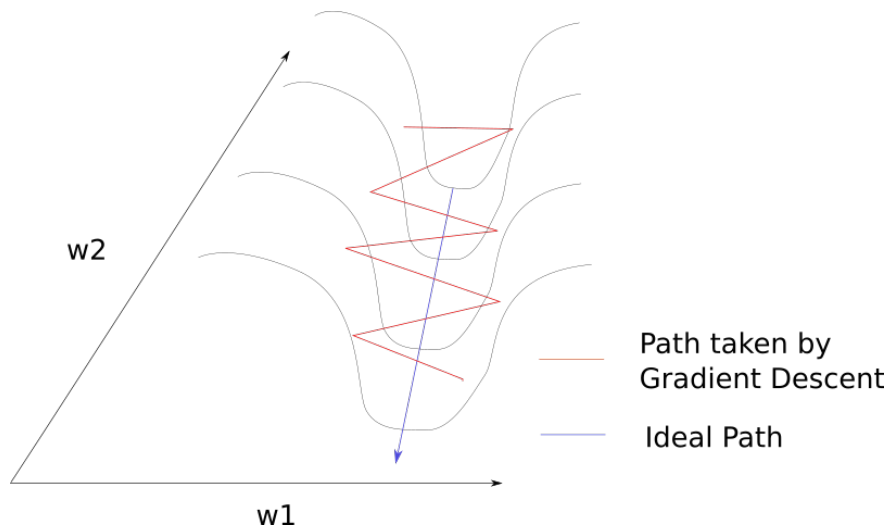


Figure 7: Gradient descent path bounces along ridges of ravine, because surface curves much more steeply in direction of w_1 .

Gradient descent in a ravine (2)

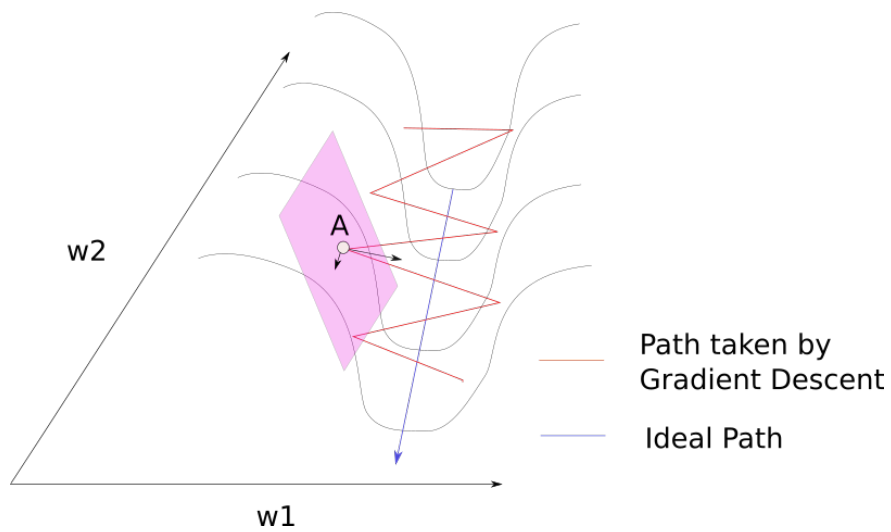


Figure 8: Gradient descent path bounces along ridges of ravine, because surface curves much more steeply in direction of w_1 .

Momentum (1)

- Idea: Update includes a *velocity* vector v , that accumulates gradient of past steps.
- Each update is a linear combination of the gradient and the previous updates.
- (Go faster if gradient keeps pointing in the same direction!)

Momentum (2)

Classical momentum: for some $0 \leq \gamma_t < 1$,

$$v^{t+1} = \gamma^t v^t + \nabla L(w^t)$$

so

$$w^{t+1} = w^t - \alpha^t v^{t+1} = w^t - \alpha^t (\gamma^t v^t + \nabla L(w^t))$$

(γ may be in range 0.9 - 0.99.)

Momentum: pseudocode

GD:

```
for t in range(num_steps):  
    dw = compute_grad(w)  
    w -= lr * dw
```

GD + Momentum:

```
for t in range(num_steps):  
    dw = compute_grad(w)  
    v = gamma * v + dw  
    w -= lr * v
```

Momentum: illustrated

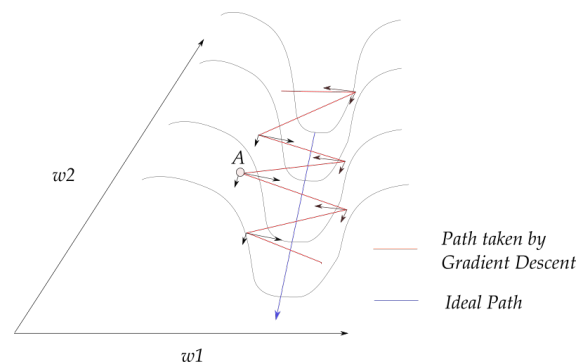


Figure 9: Momentum dampens oscillations by reinforcing the component along w_2 while canceling out the components along w_1 .

AdaGrad (1)

Next idea: “per-parameter learning rates”!

Track per-parameter square of gradient, to normalize parameter update step.

AdaGrad (2)

$$w^{t+1} = w^t - \frac{\alpha}{\sqrt{v^{t+1} + \epsilon}} \nabla L(w^t)$$

where

$$v^{t+1} = v^t + \nabla L(w^t)^2$$

Weights with large gradient have smaller learning rate, weights with small gradients have larger learning rates.

i.e.: take smaller steps in steep directions, take bigger steps where the gradient is flat.

AdaGrad: pseudocode

GD:

```
for t in range(num_steps):  
    dw = compute_grad(w)  
    w -= lr * dw
```

GD + AdaGrad:

```
grad_sq = 0  
for t in range(num_steps):  
    dw = compute_grad(w)  
    grad_sq = grad_sq + dw * dw  
    w -= lr * dw / sqrt(grad_sq + epsilon)
```

RMSProp: Leaky AdaGrad

Idea: Use EWMA to emphasize *recent* gradient magnitudes.

$$w^{t+1} = w^t - \frac{\alpha}{\sqrt{v^{t+1} + \epsilon}} \nabla L(w^t)$$

where

$$v^{t+1} = \gamma v^t + (1 - \gamma) \nabla L(w^t)^2$$

RMSProp: pseudocode

GD + AdaGrad:

```
grad_sq = 0
for t in range(num_steps):
    dw = compute_grad(w)
    grad_sq = grad_sq + dw * dw
    w -= lr * dw / sqrt(grad_sq + epsilon)
```

GD + RMSProp:

```
grad_sq = 0
for t in range(num_steps):
    dw = compute_grad(w)
    grad_sq = gamma * grad_sq + (1 - gamma) * dw * dw
    w -= lr * dw / sqrt(grad_sq + epsilon)
```

Adam: Adaptive Moment Estimation

- Uses ideas from momentum (first moment) and RMSProp (second moment)!
- plus bias correction

Adam: pseudocode vs Momentum

GD + Momentum:

```
for t in range(num_steps):
    dw = compute_grad(w)
    v = gamma * v + dw
    w -= lr * v
```

GD + Adam (without bias correction):

```
moment1 = 0
moment2 = 0
for t in range(num_steps):
    dw = compute_grad(w)
    moment1 = b1 * moment1 + (1 - b1) * dw
    moment2 = b2 * moment2 + (1 - b2) * dw * dw
    w -= lr * moment1 / sqrt(moment2 + epsilon)
```

Adam: pseudocode vs RMSProp

GD + RMSProp:

```
grad_sq = 0
for t in range(num_steps):
    dw = compute_grad(w)
    grad_sq = gamma * grad_sq + (1 - gamma) * dw * dw
    w -= lr * dw / sqrt(grad_sq + epsilon)
```

GD + Adam (without bias correction):

```
moment1 = 0
moment2 = 0
for t in range(num_steps):
    dw = compute_grad(w)
    moment1 = b1 * moment1 + (1 - b1) * dw
    moment2 = b2 * moment2 + (1 - b2) * dw * dw
    w -= lr * moment1 / sqrt(moment2 + epsilon)
```

Usually b_1 is smaller than b_2 , i.e. we update `moment1` more aggressively than `moment2`.

Adam: Pseudocode with bias correction

```
moment1 = 0
moment2 = 0
for t in range(num_steps):
    dw = compute_grad(w)
    moment1 = b1 * moment1 + (1 - b1) * dw
    moment2 = b2 * moment2 + (1 - b2) * dw * dw
    moment1_unbias = moment1 / (1 - b1 ** t)
    moment2_unbias = moment2 / (1 - b2 ** t)
    w -= lr * moment1_unbias / sqrt(moment2_unbias + epsilon)
```

When we initialize both moments to zero, they are initially “biased” to smaller values (since they update slowly!) This bias correction accounts for that.

Illustration (Beale's function)

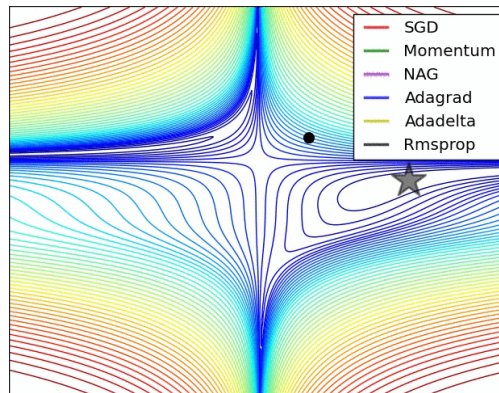


Figure 10: Animation credit: Alec Radford. [Link for animation.](#)

Due to the large initial gradient, velocity based techniques shoot off and bounce around, while those that scale gradients/step sizes like RMSProp proceed more like accelerated SGD.

Illustration (Long valley)

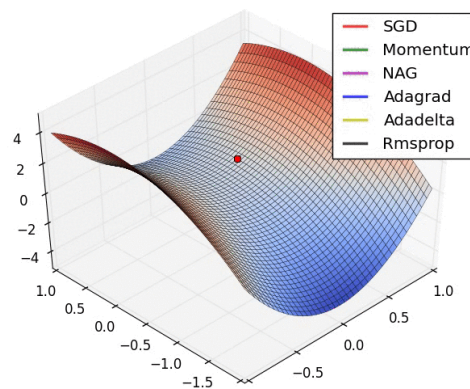


Figure 11: Animation credit: Alec Radford. [Link for animation.](#)

SGD stalls and momentum has oscillations until it builds up velocity in optimization direction. Algorithms that scale step size quickly break symmetry and descend in optimization direction.

Recap

- Gradient descent as a general approach to training
- Variations

Gradient descent is easy on linear regression! You won't get to apply any of these more advanced techniques until later in the semester, when we work with less friendly loss surfaces.